BiblioTech: passo a passo para a Avaliação Final

Nov 13, 2026 · @Mago

Visão geral

Hoje o BiblioTech atende 3 dos 12 requisitos do PDF da avaliação e 2 em parte; este guia cobre os outros 7, na ordem em que um destrava o outro. A leitura vem do código no commit `f3da7fe` (12/nov/2026). Nada foi compilado nem executado: os trechos de código são esqueletos, e a IDE vai apontar imports e ajustes pequenos.

O que o professor testa ao vivo, e que fase cobre cada teste

Teste	O que ele faz	Fase
1. Bootstrapping	Limpa o banco, abre o React (login), roda o JavaFX, cria empresa e administrador, loga	3
2. Segurança	Chama um endpoint protegido com token; desliga a segurança no código, reinicia e chama sem token	1
3. Multilocação	A cria um registro e sai; B entra e não vê; o GET por id do registro de A deve ser bloqueado	4
4. Redux e interceptor	Faz login, reinicia o back, recarrega a lista; o login aparece sozinho; mostra a funcionalidade extra no Redux	1 e 2
5. Integração externa	Cadastra um CEP e mostra o endereço preenchido pelo ViaCEP	5
6. Explicação	Camadas e portas, relacionamentos JPA, store, slice e reducers	6 e 7

Decisões fechadas

Decisão	Escolha	Por quê
Modelo de multilocação	Empresa = Biblioteca. Não existe administrador global. O bootstrap cria uma biblioteca e um ADMINISTRADOR vinculado a ela; `usuario.biblioteca_id` passa a ser `NOT NULL`	O PDF manda o JavaFX criar o primeiro Administrador **e** a primeira Empresa. A regra antiga "biblioteca nula vê tudo" era um buraco e deixa de existir
Segunda empresa (para o Teste 3)	Rodar o desktop de novo: o endpoint de bootstrap é protegido pela chave do operador e cria outra biblioteca com outro administrador	A e B ficam em empresas diferentes, e os dois já podem criar livro
Quem tem dono	`biblioteca_id` em Livro, Autor, Exemplar e Empréstimo (Usuário já tem). Editora fica fora, porque não tem controller	Regra uniforme: toda tabela de negócio tem dono, e toda consulta filtra por ele
Registro público	Removido (`/auth/registro` e a tela `cadastro`)	Um cadastro público não tem empresa. Usuários são criados pelo administrador da biblioteca
Desktop	Só JavaFX, falando com a API (não com o banco), na pasta `admin-desktop/`	Reaproveita validação de CPF, BCrypt e regras do back
Recarregar a página	Desloga	O PDF exige token e usuário "exclusivamente na store do Redux": sem cookie, sem `localStorage` , sem `redux-persist`
Integração externa	ViaCEP, na tela "Minha biblioteca"	Menos peças para dar errado na sala do que enviar e-mail

Ordem de trabalho

1. Fase 1: base do back (login devolve o usuário, expiração por `.env` , logout, tokens apagados na subida).
2. Fase 2: front com Redux, interceptors, favoritos e multilayout.
3. Fase 3: banco limpo, bootstrap e desktop JavaFX.
4. Fase 4: multilocação.
5. Fase 5: ViaCEP e "Minha biblioteca".

6. Fase 6: portas, inversão de dependência e relacionamentos JPA.
7. Fase 7: limpeza e ensaio dos 6 testes.

A Fase 3 vem antes da 4 de propósito: você recria o banco uma vez só, a migration de multilocação já nasce com `NOT NULL`, e é o bootstrap que cria as duas bibliotecas do Teste 3. A Fase 1 vem antes da 2 porque o login passa a devolver o usuário que o Redux precisa guardar.

Como trabalhar

Faça um commit por passo e rode a checagem da fase antes de seguir; assim, quando algo quebra, você sabe qual commit foi.

- **Caminhos:** os do back são relativos a `src/main/java/br/com/senac/bibliotech/` (escrevo `…/`) e, para recursos, a `src/main/resources/`. Os do front são relativos a `front/`, e `app/` é a pasta do Next.
- **Checagem ao fim de cada passo:** `./mvnw verify` no back; `npm run lint` e `npm run build` no front.
- **`pom.xml` do back:** nenhum passo precisa mexer nele (Spring Boot 4.1.1, Java 25). O desktop tem `pom.xml` próprio.
- **Banco:** vamos editar a migration V5 (tirar o seed). O Flyway recusa um banco já migrado nesse caso, então recrie com `docker compose down -v` e `docker compose up -d`.
- **`.env` (raiz, fora do Git):** `JWT_SECRET` (mínimo 32 caracteres), `JWT_EXPIRATION_MINUTES`, `ADMIN_PROMOTION_PASSWORD` e `BOOTSTRAP_KEY`. Nunca commite valores reais.
- **O que eu não verifiquei:** build, testes e execução. Nomes como `UsuarioLogado.authorities()` vêm de leitura parcial; se a IDE reclamar, ajuste ao que existe.

Cada passo abaixo diz o arquivo, o que fazer nele e, ao fim da fase, uma linha "Pronto quando" que você consegue testar na mão.

Fase 1: base do back

Esta fase faz o login devolver o usuário e o vencimento, deixa a expiração configurável, cria o logout e garante que reiniciar o back invalide os tokens, que é o exemplo que o PDF usa no Teste 4.

1. DTO do usuário logado. Crie `…/application/dto/UsuarioLogadoResponse.java`. É um DTO próprio para não vazar CPF nem senha.

```
public record UsuarioLogadoResponse(Long id, String nome, String email,
                                     EnumPerfil perfil, Long bibliotecaId) {
    public static UsuarioLogadoResponse from(Usuario u) {
        Long bibliotecaId = u.getBiblioteca() == null ? null :
u.getBiblioteca().getId();
        return new UsuarioLogadoResponse(u.getId(), u.getNome(),
u.getEmail(), u.getPerfil(), bibliotecaId);
    }
}
```

2. Login devolve o usuário. Em `…/application/dto/LoginResponse.java`, acrescente `UsuarioLogadoResponse usuario` no fim do record. Em `…/application/service/AuthService.java`, no `login()`, passe `UsuarioLogadoResponse.from(usuario)` ao construir a resposta.

3. Logout. Em `…/application/service/TokenService.java`, crie `@Transactional void revogar(String token)` com `tokenRepository.findTokenByToken(token).ifPresent(tokenRepository::delete)`. Em `AuthService`, crie `logout(String token)` que só delega. Em `…/presentation/AuthController.java`:

```
@PostMapping("/logout")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void logout(@RequestHeader("Authorization") String cabecalho) {
    authService.logout(cabecalho.substring("Bearer ".length()));
}
```

Não põe `/auth/logout` na lista pública: o `anyRequest().authenticated()` já protege.

4. Configuração. Em `application.properties`:

```
api.security.token.expiration-minutes=${JWT_EXPIRATION_MINUTES:60}
logging.level.org.springframework.security=INFO
```

No ensaio do Teste 4, ponha `JWT_EXPIRATION_MINUTES=1` no `.env`; depois volte para 60.

5. Reiniciar o back invalida os tokens. Hoje o token fica na tabela `token`, então reiniciar não invalida nada, e o exemplo do professor ("reiniciando o backend para invalidar tokens em memória") falharia. Apague a tabela quando a aplicação sobe. O custo é que toda reinicialização desloga todo mundo, o que equivale ao comportamento de um token só em memória. Crie `…/infrastructure/config/LimpaTokensNaSubida.java`:

```
@Component
@RequiredArgsConstructor
class LimpaTokensNaSubida implements ApplicationRunner {
    private final TokenRepository tokenRepository;

    @Override
    @Transactional
    public void run(ApplicationArguments args) {
        tokenRepository.deleteAllInBatch();
    }
}
```

6. Endpoint de demonstração do Teste 2. Com a segurança desligada, qualquer endpoint que filtra pela biblioteca do logado devolve 500, porque não há usuário. Use uma lista que não dependa dele. Em `…/presentation/LivroController.java`:

```
@GetMapping("/generos")
public List<EnumGeneroLiterario> generos() {
    return List.of(EnumGeneroLiterario.values());
}
```

Ele cai no `anyRequest().authenticated()`, então é protegido, e o front aproveita para montar o seletor de gêneros. Ele só serve se o `EnumGeneroLiterario` estiver preenchido. Não torne o filtro de multilocação tolerante a usuário ausente: isso devolveria tudo a quem não está logado.

Pronto quando:

- `POST /auth/login` no Swagger devolve `usuario` junto com o token, e com `JWT_EXPIRATION_MINUTES=1` o `expiraEm` vem 1 minuto à frente.
- `POST /auth/logout` com o token devolve 204, e uma chamada logo depois com o mesmo token devolve 401.
- Reiniciar o back e repetir uma chamada com um token antigo devolve 401.
- `GET /livros/generos` com token devolve a lista; sem token, 401.

Fase 2: front com Redux

O PDF pede token e usuário só na store, interceptor de requisição lendo a store, interceptor de resposta despachando logout no 401, uma segunda funcionalidade no Redux e layouts diferentes para login e painel. A Fase 1 precisa estar pronta, porque o login passa a devolver o usuário.

2.1 Token, vencimento e usuário só na store (sem cookie)

Arquivo	O que fazer
`app/types/auth.ts`	Acrescente `UsuarioLogado` (`id`, `nome`, `email`, `perfil`, `bibliotecaId: number | null`) e faça `LoginResponse` ser `{ token, tipo, expiraEm, usuario: UsuarioLogado }`
`app/store/authSlice.ts`	Substitua o arquivo inteiro pelo código abaixo
`app/services/api.ts`	Tire os imports de `js-cookie` e `./sessao`; troque os dois interceptors pelos abaixo e exporte `configurarApi`. Mantenha o bloco que lê `corpo.detail` e monta `new Error(mensagem)`
`app/store/store.ts`	Chame `configurarApi(store);` logo depois do `configureStore`
`app/services/authService.ts`	Acrescente `logoutNoServidor()` (código abaixo)
`app/components/ExpiracaoDoToken.tsx` (novo)	Código abaixo
`app/components/StoreProvider.tsx`	Código abaixo
`app/components/TelaLogin.tsx` (novo)	Mova para cá o formulário de `app/login/page.tsx` e troque o `entrar()` pelo abaixo
`app/(sistema)/layout.tsx`	Código abaixo
`app/components/Header.tsx`	Troque o `sair()` pelo abaixo; remova `useRouter`, `limparSessao` e `sessaoEncerrada`
`usuarios/page.tsx`, `usuarios/[id]/editar/page.tsx`, `livros/[id]/page.tsx` (em `app/(sistema)/`)	Troque `obterEmailLogado()` por `useAppSelector((s) => s.auth.usuario)`; o perfil já está na store, não precisa mais procurar o e-mail na lista de usuários
`app/cadastro/`	Apague a pasta inteira (não há registro público) e tire os links para `/cadastro` do `TelaLogin` e da landing

Arquivo	O que fazer
`app/services/sessao.ts`	Apague
terminal	`cd front && npm uninstall js-cookie @types/js-cookie`

O slice nasce vazio e `login` devolve o estado inteiro. Há um único reducer de saída, e botão Sair, timer e 401 passam todos por ele.

```
// store/authSlice.ts
import { createSlice, type PayloadAction } from "@reduxjs/toolkit";

export interface Usuario { id: number; nome: string; email: string; perfil:
string; bibliotecaId: number | null }
export interface AuthState { token: string | null; expiraEm: string | null;
usuario: Usuario | null }

const initialState: AuthState = { token: null, expiraEm: null, usuario: null
};

const authSlice = createSlice({
  name: "auth",
  initialState,
  reducers: {
    login: (_state, action: PayloadAction<AuthState>) => action.payload,
    logout: () => initialState,
  },
});

export const { login, logout } = authSlice.actions;
export default authSlice.reducer;
```

O interceptor lê a store, nunca o cookie. A store é injetada por função para evitar import circular (`import type` some na compilação).

```
// services/api.ts
import type { store } from "../store/store";
import { logout } from "../store/authSlice";

let storeRef: typeof store;
export function configurarApi(s: typeof store) { storeRef = s; }

api.interceptors.request.use((config) => {
  const token = storeRef.getState().auth.token;
  if (token) config.headers.Authorization = `Bearer ${token}`;
```

```
  return config;
});

api.interceptors.response.use(
  (resposta) => resposta,
  (erro) => {
    // Só desloga se HAVIA sessão: o 401 de senha errada no /auth/login não
pode derrubar nada.
    // 403 (falta de perfil) não desloga: o token é válido.
    if (erro.response?.status === 401 && storeRef.getState().auth.token) {
      storeRef.dispatch(logout());
    }
    // ...mantenha aqui o bloco que monta new Error(mensagem) a partir do
ProblemDetail
    return Promise.reject(erro);
  },
);
```

```
// services/authService.ts
export async function logoutNoServidor(): Promise<void> {
  try { await api.post("/auth/logout"); } catch { /* token já inválido: tudo
bem */ }
}
```

Um timer desloga no instante do vencimento, mesmo sem nenhuma requisição. É isso que faz o login aparecer sozinho com `JWT_EXPIRATION_MINUTES=1`.

```
// components/ExpiracaoDoToken.tsx
"use client";
import { useEffect } from "react";
import { useAppDispatch, useAppSelector } from "../store/hooks";
import { logout } from "../store/authSlice";

export default function ExpiracaoDoToken() {
  const dispatch = useAppDispatch();
  const expiraEm = useAppSelector((s) => s.auth.expiraEm);

  useEffect(() => {
    if (!expiraEm) return;
    const venceEm = new Date(expiraEm).getTime();
    const verificar = () => { if (Date.now() >= venceEm) dispatch(logout());
};
    const timer = setTimeout(verificar, Math.max(venceEm - Date.now(), 0));
    // Aba em segundo plano tem timer atrasado: ao voltar, confere de novo.
    document.addEventListener("visibilitychange", verificar);
    return () => { clearTimeout(timer);
```

```
document.removeEventListener("visibilitychange", verificar); };
  }, [expiraEm, dispatch]);

  return null;
}
```

```
// components/StoreProvider.tsx
"use client";
import { Provider } from "react-redux";
import { store } from "../store/store";
import ExpiracaoDoToken from "./ExpiracaoDoToken";

export default function StoreProvider({ children }: { children:
React.ReactNode }) {
  // Sem restauração de cookie: a store nasce vazia, então recarregar a
página desloga (decisão fechada).
  return (
    <Provider store={store}>
      <ExpiracaoDoToken />
      {children}
    </Provider>
  );
}
```

O `entrar()` do `TelaLogin` grava tudo na store. O service e a ação têm o mesmo nome (`login`), então renomeie um deles no import.

```
import { login as loginNoBack } from "@/app/services/authService";
import { login as sessaoIniciada } from "../store/authSlice";

const pathname = usePathname(); // de next/navigation

async function entrar(e: React.FormEvent) {
  e.preventDefault();
  setErro(""); setCarregando(true);
  try {
    const r = await loginNoBack(email, senha);
    dispatch(sessaoIniciada({ token: r.token, expiraEm: r.expiraEm, usuario:
r.usuario }));
    // Se o login foi mostrado pelo layout do painel (token vencido), o
layout se redesenha sozinho.
    if (pathname === "/login") router.push("/home");
  } catch (err) {
    setErro(err instanceof Error ? err.message : "Falha ao entrar");
  } finally {
    setCarregando(false);
  }
}
```

O layout do painel observa o token e renderiza o login quando ele some, sem `window.location` nem `router.push`. Isso segue o texto do PDF ("renderizar condicionalmente a tela de login") e ainda elimina o `useEffect` com `setState` que dá o erro de ESLint neste arquivo. O custo: a URL fica em `/livros` enquanto o login aparece.

```
// app/(sistema)/layout.tsx
"use client";
import { useState } from "react";
import Header from "../components/Header";
import Sidebar from "../components/Sidebar";
import Footer from "../components/Footer";
import TelaLogin from "../components/TelaLogin";
import { useAppSelector } from "../store/hooks";

export default function SistemaLayout({ children }: { children:
React.ReactNode }) {
  const token = useAppSelector((s) => s.auth.token);
  const [aberta, setAberta] = useState(true); // hooks ANTES do return
condicional

  if (!token) return <TelaLogin />;

  return (
```

```
    <div className="flex min-h-screen flex-col">
      <Header onToggle={() => setAberta(!aberta)} />
      <div className="flex flex-1">
        <Sidebar aberta={aberta} />
        <main className="flex-1 p-6">{children}</main>
      </div>
      <Footer />
    </div>
  );
}
```

```
// Header.tsx
const usuario = useAppSelector((e) => e.auth.usuario);
const email = usuario?.email ?? "";

async function sair() {
  await logoutNoServidor(); // apaga a linha da tabela token (erro ignorado)
  dispatch(logout());       // a store esvazia e o layout mostra o login
}
```

Pronto quando: `npm run lint` e `npm run build` passam sem nenhum `js-cookie`; ao logar, o DevTools mostra Cookies e `localStorage` sem token; recarregar a página volta ao login; com `JWT_EXPIRATION_MINUTES=1` o login aparece sozinho; reiniciar o back e recarregar a lista faz o 401 virar login.

2.2 Segunda funcionalidade no Redux: favoritos de livros

O PDF pede Redux para a autenticação e para mais uma funcionalidade. Na apresentação você mostra a estrela funcionando e explica store, slice e reducers com ela.

```
// app/store/favoritosSlice.ts (novo)
import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
import { logout } from "./authSlice";

interface FavoritosState { ids: number[] }
const initialState: FavoritosState = { ids: [] };

const favoritosSlice = createSlice({
  name: "favoritos",
  initialState,
  reducers: {
    adicionarFavorito(state, action: PayloadAction<number>) {
      // Immer: aqui "mutar" o state é seguro, ele gera um estado novo por
baixo.
      if (!state.ids.includes(action.payload))
```

```
state.ids.push(action.payload);
    },
    removerFavorito(state, action: PayloadAction<number>) {
      state.ids = state.ids.filter((id) => id !== action.payload);
    },
    limparFavoritos: () => initialState,
  },
  // Ao sair, os favoritos também somem: a store é uma só no navegador.
  extraReducers: (builder) => { builder.addCase(logout, () => initialState);
},
});

export const { adicionarFavorito, removerFavorito, limparFavoritos } =
favoritosSlice.actions;
export default favoritosSlice.reducer;
```

Em `app/store/store.ts`, use `reducer: { auth: authReducer, favoritos: favoritosReducer }`. Em `app/(sistema)/livros/page.tsx`:

```
const dispatch = useAppDispatch();
const favoritos = useAppSelector((s) => s.favoritos.ids);
const [soFavoritos, setSoFavoritos] = useState(false);

// 1) no filter que já existe, como primeira linha:
if (soFavoritos && !favoritos.includes(l.id)) return false;

// 2) acima da grade, junto da busca:
<label className="mb-4 mr-4 inline-flex items-center gap-2">
  <input type="checkbox" checked={soFavoritos} onChange={(e) =>
setSoFavoritos(e.target.checked)} />
  Só favoritos ({favoritos.length})
</label>
<button type="button" onClick={() => dispatch(limparFavoritos())}>Limpar
favoritos</button>

// 3) no .map: a estrela fica AO LADO do Link (botão dentro de <a> é HTML
inválido)
{filtrados.map((l) => {
  const favorito = favoritos.includes(l.id);
  return (
    <div key={l.id} className="relative">
      <Link href={`/livros/${l.id}`} className="...classes que o Link já
tem...">{/* card atual */}</Link>
      <button
        type="button"
        aria-label={favorito ? "Remover dos favoritos" : "Adicionar aos
```

```
favoritos"}
        onClick={() => dispatch(favorito ? removerFavorito(l.id) :
adicionarFavorito(l.id))}
        className="absolute right-2 top-2 z-10 text-3xl"
      >
        {favorito ? "★" : "☆"}
      </button>
    </div>
  );
})}
```

Pronto quando: você favorita dois livros, marca "Só favoritos" e vê só eles; limpa e a lista zera; sai, entra de novo e as estrelas voltam vazias.

2.3 Multilayout e telas por perfil

A hierarquia que você explica no Teste 6: `app/layout.tsx` (fontes e `StoreProvider`) contém `(publico)/layout.tsx` (sem menu) ou `(sistema)/layout.tsx` (cabeçalho, menu e rodapé, com o login no lugar quando não há token).

```
# Pasta com parênteses é um "grupo de rotas": organiza layouts sem entrar na
URL.
# /login e / continuam com o mesmo endereço. Use aspas, por causa dos
parênteses.
cd front/app
mkdir "(publico)"
git mv login "(publico)/login"
git mv page.tsx "(publico)/page.tsx"
```

- `app/(publico)/layout.tsx` (novo): `export default function PublicoLayout({ children }: { children: React.ReactNode }) { return <div className="min-h-screen">{children}</div>; }`, sem Header nem Sidebar.
- `app/(publico)/page.tsx` (a landing, movida): troque os imports relativos `./components/...` por `@/app/components/...`.
- `app/login/page.tsx` (agora em `(publico)/login/`): fica só com `return <TelaLogin />`.

O bloqueio por perfil protege a **rota**, não só o item de menu. Quem protege de verdade é o back (403 no `SecurityConfiguration`); isto é conforto de interface.

```
// app/components/SomentePerfis.tsx (novo)
"use client";
import { useAppSelector } from "../store/hooks";

export default function SomentePerfis({ perfis, children }: { perfis:
string[]; children: React.ReactNode }) {
  const perfil = useAppSelector((s) => s.auth.usuario?.perfil);
  if (!perfil || !perfis.includes(perfil)) {
    return <p className="mx-auto max-w-xl p-6 text-lg">Acesso restrito.</p>;
  }
  return <>{children}</>;
}
```

- `app/(sistema)/usuarios/layout.tsx` (novo): `<SomentePerfis perfis={["ADMINISTRADOR", "BIBLIOTECARIO"]}>{children}</SomentePerfis>`.
- `app/(sistema)/minha-biblioteca/layout.tsx` (novo): o mesmo com `perfis={["ADMINISTRADOR"]}` (a tela nasce na Fase 5).

```
// app/components/Sidebar.tsx
const itens = [
  { href: "/home", icone: "🏛️", nome: "Início" },
  { href: "/usuarios", icone: "👥", nome: "Usuários", perfis:
["ADMINISTRADOR", "BIBLIOTECARIO"] },
  { href: "/livros", icone: "📖", nome: "Livros" },
  { href: "/autores", icone: "🖋️", nome: "Autores" },
  { href: "/emprestimos", icone: "📜", nome: "Empréstimos" },
  { href: "/minha-biblioteca", icone: "🏰", nome: "Minha biblioteca", perfis:
["ADMINISTRADOR"] },
];

const perfil = useAppSelector((s) => s.auth.usuario?.perfil);
const visiveis = itens.filter((i) => !i.perfis || (perfil &&
i.perfis.includes(perfil)));
// e no JSX, troque itens.map por visiveis.map
```

Pronto quando: `/` e `/login` abrem sem menu; após logar, o painel mostra cabeçalho e menu; logado como leitor, o menu não mostra `Usuários` nem `Minha biblioteca`, e digitar `/usuarios` mostra "Acesso restrito".

Fase 3: banco limpo, bootstrap e desktop JavaFX

Esta fase tira todo `INSERT` das migrations, cria o endpoint que provisiona uma biblioteca junto com o seu administrador e entrega o app JavaFX que o chama. É o Teste 1 do PDF ("sem a necessidade de INSERTs manuais").

3.1 Tirar o seed

Em `…/resources/db/migration/V5__adicionar_colunas_emprestimo_e_seed_data.sql`, apague o bloco de seed: do cabeçalho "SEED DE DADOS" (por volta da linha 46) até antes da seção "NOT NULL condicional" (por volta da 122). São seis `INSERT`: editora, autor, livro, usuario (inclui o `admin@bibliotech.com`), leitor e exemplar. Mantenha as seções de estrutura. Os `UPDATE` da V6 viram inofensivos em tabelas vazias.

Confira com `grep -n "INSERT INTO" src/main/resources/db/migration/*.sql`: não deve sobrar nada. Depois recrie o banco:

```
docker compose down -v   # -v apaga o volume do Postgres
docker compose up -d
```

3.2 Endpoint `POST /auth/bootstrap`

Ele é público, mas protegido por uma chave do operador que vem do `.env`. Cada chamada cria uma biblioteca e o seu administrador, os dois na mesma transação. Chave vazia no `.env` = bootstrap desligado.

Arquivo	O que fazer
`application.properties`	`app.bootstrap.chave=${BOOTSTRAP_KEY:}`
`.env`	`BOOTSTRAP_KEY=` com uma chave longa e aleatória (por exemplo, a saída de `openssl rand -hex 32`)
`…/application/exception/AcessoNegadoException.java` (novo)	`extends RuntimeException`
`…/presentation/ApiExceptionHandler.java`	`@ExceptionHandler(AcessoNegadoException.class)` devolvendo `HttpStatus.FORBIDDEN`, igual aos handlers que já existem
`…/application/dto/BootstrapRequest.java` (novo)	Record com `empresaNome`, `empresaCnpj`, `adminNome`, `adminEmail`, `adminCpf`, `adminSenha`, com as mesmas validações do `UsuarioRequest` (senha de 8 a 72 caracteres, CPF com 11 dígitos). A chave **não** vai no corpo: vai no header
`…/application/dto/BootstrapResponse.java` (novo)	`Long bibliotecaId, String bibliotecaNome, Long usuarioId, String email`
`…/domain/repository/BibliotecaRepository.java`	`boolean existsByCnpj(String cnpj);`
`…/application/service/BootstrapService.java` (novo)	Código abaixo

Arquivo	O que fazer
`…/presentation/AuthController.java`	Endpoint abaixo
`…/infrastructure/security/SecurityConfiguration.java`	Acrescente `"/auth/bootstrap"` no `permitAll()`
`…/infrastructure/security/JwtFilter.java`	No `shouldNotFilter`, acrescente `uri.equals("/auth/bootstrap")` ao lado do `/auth/login`

A ordem das regras importa: a chave vem primeiro, para quem não a conhece receber 403 sem descobrir nada sobre o sistema.

```
@Service
@RequiredArgsConstructor
public class BootstrapService {

    private final BibliotecaRepository bibliotecaRepository;
    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;

    @Value("${app.bootstrap.chave:}")
    private String chaveConfigurada;

    @Transactional
    public BootstrapResponse provisionar(String chaveInformada,
BootstrapRequest r) {
        if (!chaveConfere(chaveInformada)) {
            throw new AcessoNegadoException("Chave de bootstrap inválida");
        }

        CPF cpf = new CPF(r.adminCpf());                        // lança se os
dígitos não fecham
        String email = r.adminEmail().trim().toLowerCase(Locale.ROOT);

        if (usuarioRepository.existsByEmail(email))
            throw new ConflitoException("Já existe um usuário com este e-
mail");
        if (usuarioRepository.existsByCpf_Cpf(cpf.getCpf()))
            throw new ConflitoException("Já existe um usuário com este CPF");
        if (bibliotecaRepository.existsByCnpj(r.empresaCnpj()))
            throw new ConflitoException("Já existe uma biblioteca com este
CNPJ");

        Biblioteca biblioteca = new Biblioteca();
        biblioteca.setNome(r.empresaNome().trim());
```

```
        biblioteca.setCnpj(r.empresaCnpj());
        bibliotecaRepository.save(biblioteca);

        Usuario admin = Usuario.builder()
                .nome(r.adminNome().trim())
                .email(email)
                .cpf(cpf)
                .senha(passwordEncoder.encode(r.adminSenha()))
                .perfil(EnumPerfil.ADMINISTRADOR)
                .status(EnumStatusUsuario.ATIVO)
                .biblioteca(biblioteca)
                .build();
        usuarioRepository.save(admin);

        return new BootstrapResponse(biblioteca.getId(),
biblioteca.getNome(), admin.getId(), admin.getEmail());
    }

    // Comparação em tempo constante; chave não configurada = bootstrap
desligado.
    private boolean chaveConfere(String informada) {
        if (chaveConfigurada.isBlank() || informada == null) return false;
        return
MessageDigest.isEqual(informada.getBytes(StandardCharsets.UTF_8),

chaveConfigurada.getBytes(StandardCharsets.UTF_8));
    }
}
```

Os nomes dos campos do `Usuario` (como o `cpf`, que é um value object) vêm de leitura parcial; ajuste ao que a IDE mostrar. No controller, o header é opcional de propósito: sem ele a resposta tem que ser 403 como qualquer chave errada, e não 400.

```
@PostMapping("/bootstrap")
@ResponseStatus(HttpStatus.CREATED)
public BootstrapResponse bootstrap(
        @RequestHeader(value = "X-Bootstrap-Key", required = false) String
chave,
        @Valid @RequestBody BootstrapRequest request) {
    return bootstrapService.provisionar(chave, request);
}
```

3.3 Tirar o registro público

- No `AuthController`, apague `registrar(...)` e o `@PostMapping("/registro")`.

- Tire `"/auth/registro"` do `permitAll()` do `SecurityConfiguration` e do `shouldNotFilter` do `JwtFilter`.
- `UsuarioService.cadastrar` continua existindo: é ele que serve ao `POST /usuarios` (administrador e bibliotecário).

3.4 Como subir o projeto do zero

Sem isso o professor não consegue rodar nada. Crie na raiz:

- **`.env.example`**, com as chaves e sem valores: `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET` (mínimo 32 caracteres: o `TokenService` recusa menos), `JWT_EXPIRATION_MINUTES`, `ADMIN_PROMOTION_PASSWORD`, `BOOTSTRAP_KEY`.
- **`README.md`**, com passos numerados: copiar `.env.example` para `.env` e preencher; `docker compose up -d`; `./mvnw spring-boot:run`; `cd front && npm install && npm run dev`; abrir o app desktop e criar a biblioteca com o administrador; logar no front.

3.5 App desktop em JavaFX

O app falando com a API (e não com o banco) reaproveita a validação de CPF, o BCrypt e as regras do back. Você já tem a base na pasta `back/` (a que corrigimos antes). Ajuste assim:

1. **Renomeie a pasta** para não confundir com o back em Spring: `git mv back admin-desktop`.
2. **`Config.java`:** fique só com `apiUrl()` (variável `BIBLIOTECH_API_URL`, padrão `http://localhost:8080`). Apague `adminKey()` e `bootstrapKey()`.
3. **Tela 1 (a chave do operador):** troque a comparação local pela simples guarda da chave digitada em memória e siga para a tela 2. Quem valida a chave é o back; uma checagem local seria só enfeite, porque qualquer um extrai a string de um `.class`.
4. **Tela 2 (primeira configuração):** dois grupos de campos, **Empresa** (nome, CNPJ) e **Administrador** (nome, e-mail, CPF, senha). O CPF e o CNPJ seguem aceitando máscara: o app tira o que não é dígito antes de enviar.
5. **Chamada à API**, fora da thread da interface (`Task`), como no `UsuarioController` atual, trocando o endereço, o header e o corpo:

```
String json = "{"
    + "\"empresaNome\":" + aspas(empresaNome) + ","
    + "\"empresaCnpj\":" + aspas(cnpj) + ","
    + "\"adminNome\":" + aspas(nome) + ","
    + "\"adminEmail\":" + aspas(email) + ","
    + "\"adminCpf\":" + aspas(cpf) + ","
    + "\"adminSenha\":" + aspas(senha) + "}";

HttpRequest request = HttpRequest.newBuilder(URI.create(Config.apiUrl() +
"/auth/bootstrap"))
        .timeout(Duration.ofSeconds(10))
        .header("Content-Type", "application/json; charset=UTF-8")
        .header("X-Bootstrap-Key", Sessao.chave())
        .POST(HttpRequest.BodyPublishers.ofString(json,
StandardCharsets.UTF_8))
        .build();
```

`Sessao` é uma classe de duas linhas com a chave em um campo estático. O método `aspas(...)` (escape para JSON) já existe no `UsuarioController` que te mandei.

6. **Respostas:** 201 mostra "Biblioteca X criada com o administrador Y" e limpa o formulário (para você poder cadastrar a segunda empresa sem reabrir o app); 403, "Chave inválida"; 409 e 400, o texto do campo `detail` da resposta; falha de conexão, "Não foi possível falar com a API em …".
7. **Menu:** a tela de menu deixa de ser necessária (chave, formulário). Apague `MenuController` e `menu-view.fxml`, ou deixe de lado.
8. **Rodar:** `cd admin-desktop && mvn clean javafx:run` (JDK 21 ou mais; a `mainClass` já aponta para `AdminApplication`).

Pronto quando:

- Você roda `docker compose down -v` e `up -d`, sobe o back e o front (login), cria a biblioteca e o administrador pelo desktop e consegue logar com ele no front.
- Repetir com o mesmo CNPJ ou e-mail devolve 409; com a chave errada, 403.
- Rodar o desktop de novo com outro CNPJ, e-mail e CPF cria a segunda biblioteca, que o Teste 3 vai usar.

Fase 4: multilocação (Teste 3)

Regra do Teste 3: o administrador da biblioteca A não enxerga nem mexe em nada da B, e o contrário também. Como a biblioteca é a "empresa", cada registro de negócio pertence a uma biblioteca, e o back descobre qual é pelo token, nunca pelo corpo da requisição.

4.1 Migration V9

Crie `V9__multilocacao.sql`. Como o banco agora nasce vazio (Fase 3), dá para exigir `NOT NULL` direto, sem backfill. Se você já tem dados de teste, recrie o banco com `down -v`.

```
ALTER TABLE livro      ADD COLUMN biblioteca_id BIGINT NOT NULL REFERENCES
biblioteca(id);
ALTER TABLE autor      ADD COLUMN biblioteca_id BIGINT NOT NULL REFERENCES
biblioteca(id);
ALTER TABLE exemplar   ADD COLUMN biblioteca_id BIGINT NOT NULL REFERENCES
biblioteca(id);
ALTER TABLE emprestimo ADD COLUMN biblioteca_id BIGINT NOT NULL REFERENCES
biblioteca(id);
ALTER TABLE usuario    ALTER COLUMN biblioteca_id SET NOT NULL;

CREATE INDEX idx_livro_biblioteca      ON livro(biblioteca_id);
CREATE INDEX idx_autor_biblioteca      ON autor(biblioteca_id);
CREATE INDEX idx_exemplar_biblioteca   ON exemplar(biblioteca_id);
CREATE INDEX idx_emprestimo_biblioteca ON emprestimo(biblioteca_id);
CREATE INDEX idx_usuario_biblioteca    ON usuario(biblioteca_id);

-- unicidade de negócio passa a ser por biblioteca (se existir UNIQUE global
no isbn, troque):
ALTER TABLE livro DROP CONSTRAINT IF EXISTS livro_isbn_key;
ALTER TABLE livro ADD CONSTRAINT uk_livro_biblioteca_isbn UNIQUE
(biblioteca_id, isbn);
```

Confira os nomes reais das tabelas e do constraint do ISBN no `\d livro` do Postgres (`docker compose exec <serviço-do-banco> psql -U <usuário> <banco>`); os nomes acima são os esperados, não conferidos no seu banco. **Editora fica de fora**: é catálogo compartilhado.

4.2 Entidades

Em `Livro`, `Autor`, `Exemplar` e `Emprestimo` acrescente:

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "biblioteca_id", nullable = false, updatable = false)
private Biblioteca biblioteca;
```

`updatable = false` impede que um registro mude de biblioteca depois de criado. Nenhum DTO de requisição ganha campo `bibliotecaId`.

4.3 Quem é o usuário logado e qual é a biblioteca dele

Você já tem a peça que devolve a biblioteca do logado (o `bibliotecaId()` do `UsuarioService`). Extraia para um componente único, `BibliotecaDoLogado`, e **apague o ramo "biblioteca nula vê tudo"** do `UsuarioService.listar()`: não existe mais administrador global, então toda consulta filtra.

```
@Component
@RequiredArgsConstructor
public class BibliotecaDoLogado {
    private final UsuarioLogadoPort logado;           // criado na Fase 6;
até lá, use o que já existe

    public Long id() {
        return logado.usuario().getBiblioteca().getId();
    }
}
```

Se você ainda não fez a Fase 6, mantenha a implementação atual (via `SecurityContextHolder`) escondida atrás dessa classe. Na Fase 6 você só troca o miolo, e os serviços não mudam.

4.4 Repositórios e serviços

Para cada entidade (`Livro`, `Autor`, `Exemplar`, `Emprestimo`, `Usuario`):

```
List<Livro> findAllByBiblioteca_Id(Long bibliotecaId);
Optional<Livro> findByIdAndBiblioteca_Id(Long id, Long bibliotecaId);
```

Nos serviços, **todo** `buscarPorId`, `atualizar` e `excluir` passam por um método único:

```
private Livro buscarNoEscopo(Long id) {
    return livroRepository.findByIdAndBiblioteca_Id(id,
bibliotecaDoLogado.id())
            .orElseThrow(() -> new RecursoNaoEncontradoException("Livro não
encontrado"));
}
```

Por que 404 e não 403: avisar "existe, mas é de outra biblioteca" já vaza informação. Para quem consulta, o recurso da outra biblioteca simplesmente não existe.

Regras de criação:

- Ao criar livro, autor, exemplar, empréstimo ou usuário, o serviço faz `setBiblioteca(bibliotecaRepository.getReferenceById(bibliotecaDoLogado.id()))`. Se o corpo trouxer qualquer outro campo de biblioteca, ele é ignorado (não

existe no DTO).

- Relacionamentos cruzados também passam pelo escopo: ao criar um exemplar para o `livroId` X, busque o livro com `buscarNoEscopo`. Senão dá para criar um exemplar da biblioteca A apontando para um livro da B. Idem empréstimo (exemplar e leitor).
- `GET` e `PUT /usuarios/{id}`: hoje o `GET /usuarios/{id}` não tem verificação nenhuma. Use `findByIdAndBiblioteca_Id` nos dois. O `PUT` já exige administrador ou o próprio usuário, e continua, agora dentro do escopo.
- Ao verificar duplicidade de ISBN, verifique **dentro da biblioteca** (`existsByBiblioteca_IdAndIsbn`).
- Listagens paginadas ou com filtro: o filtro de biblioteca entra sempre, antes de qualquer outro.

4.5 Teste automatizado

Crie `MultilocacaoTest` no serviço de livros com Mockito (ou `@SpringBootTest` se você já tem Testcontainers; sem isso, Mockito é mais rápido). O caso principal:

```
@Test
void naoEncontraLivroDeOutraBiblioteca() {
    when(bibliotecaDoLogado.id()).thenReturn(1L);
    when(livroRepository.findByIdAndBiblioteca_Id(99L,
1L)).thenReturn(Optional.empty());

    assertThrows(RecursoNaoEncontradoException.class, () ->
livroService.buscarPorId(99L));
}
```

O teste que importa de verdade é o manual abaixo; o unitário serve para o professor ver que você pensou em regressão.

4.6 Roteiro do Teste 3 (ensaie assim)

1. Com o desktop, crie as bibliotecas **A** e **B** (cada uma com seu administrador).
2. Logue como A no front, cadastre o livro "Dom Casmurro" e um usuário bibliotecário. Anote o `id` do livro (veja na resposta do Swagger ou na aba Network).
3. Logue como B: a lista de livros vem **vazia** e a de usuários mostra só os da B.
4. Ainda como B, no Swagger (com o token da B), chame `GET /livros/{id}` com o id do livro da A: **404**. Tente `PUT` e `DELETE` no mesmo id: **404** nos dois.
5. Cadastre na B um livro com o mesmo ISBN do livro da A: **cria normalmente** (unicidade por biblioteca).
6. Volte como A e confirme que o livro da B não aparece.

Pronto quando: os seis passos acima acontecem exatamente assim e `MultilocacaoTest` passa. Rode também `./mvnw test` inteiro: se algum teste antigo dependia do seed, ele quebra aqui, e é melhor descobrir agora do que na frente do professor.